Objectives

The purpose of this lab is to learn more about behavior-based animation techniques by implementing algorithms such as spatial agent vision, movement, and interaction.

Collaboration and Piazza

Please read the course policy on Academic Misconduct in the syllabus. You may discuss the lab with your classmates ONLY at a high level. You must formulate your own solution (all code must be authored by you alone) without any help from others or third party sources (e.g., the internet). Do not post any part of your solution or spoilers on Piazza.
In your lab report (see below), you need to list with whom you have discussed the lab.

I highly suggest you define sprints and work on this lab consistently rather than trying to accomplish it close to the deadline.

Assignment

Please read: Your solution must consist of source code originally authored by you alone. I would like to avoid any academic misconduct issues. Though, you may use help from video's explaining key concepts.
Use lecture material we have covered so far including collision detection, collision response, and behavior (also see **https://www.youtube.com/watch?v=CdSzlp0OSzk**). I have also provided supplemental help in a Powerpoint presentation.

The general task is to implement a 3D animation of agent's motion in two dimensions (in other words, use 3D GameObjects or models but only have the agents move in the *x* and *z* dimensions, then place a camera above the scene to give a top down/bird's eye view). The scene will contain two types of behaviors: **predator** and **prey** (i.e., **https://www.youtube.com/watch?v=CdSzlp0OSzk**). The target behavior you should try to implement is that prey wander about the scene and when a predator sees prey the predator chases it and the prey flees.

Use the "3D Core (Built-in Render Pipeline)" project (as you did in Labs #1 - #3). Do **NOT** use the URP project for you solution.

Code design: You should **DESIGN** your code well (refer to Lab #1). Start by implementing two classes, one for a predator agent and the other for a prey agent. Combine these into a class hierarchy. At a minimum, each should have a current position (x, 0, z) and heading (either an angle or vector). You may use forces to control the motion of the creatures. Give the two groups different capabilities (vision, acceleration, turning speed, etc.) to make the resulting animation interesting. Whenever a predator gets within epsilon of a prey (i.e. close or collides with it), the prey is eaten (disappears). You should

include spatial limits on the environment (i.e. do not let the creatures outside of an axis aligned box).
Your code design should allow for easy addition of other behaviors. You may want to utilize ideas from Finite State Machines (https://www.youtube.com/watch?v=JyF0oyarz4U), Behavior Trees (https://www.youtube.com/watch?v=6VBCXvfNlCM), and the State Pattern (https://www.youtube.com/watch?v=nnrOhb5UdRc).

Testing during development: Write your code incrementally. First implement movement behavior for one or two prey characters. Next implement avoidance of obstacles and other characters. To test predator and prey you may find it useful to have a player-controlled agent where in one scenario the agent has a predator behavior, and the prey interact with it. Then in another scenario the agent has a prey behavior, and the predators interact with it.

It will be faster to implement motion (translation) and collision detection (bounding box/sphere tests) logic manually than to get an understanding for all of the syntax and settings for Unity. You may use Unity components such as Triggers, Colliders, and RigidBodies (although they shouldn't be needed).

Grading

You are **NOT** allowed to use the "Navigation System" in Unity, e.g. "AI Navigation", and NavMesh.

Ideally, your solution should be such that we can run your animation and tell what you've implemented just by looking at it.

- Scene (**8 points**): Create an environment with obstacles that the creatures cannot move through. Implement one of these options.
    - (3 points/8 points) – Scene with no obstacles.
    - (6 points/8 points) – Scene with obstacles where there are some open areas, some "dead-ends", and some choke-points.
    - (8 points/8 points) – Scene with added procedurally animated set of moving obstacles.
- Agent motion (**40 points**): Use collision detection and bounding volume
    - (10 points/40 points) - Predators and prey both wander randomly
    - (30 points/40 points) - Predators chase prey when detected, prey ignore predators
    - (40 points/40 points) - Predators chase prey when detected, prey try to avoid predators
- Agent vision (**32 points**)
    - (10 points/32 points) - Omniscience (see all objects) for predators and for prey
    - (12 points/32 points) - Proximity sense (see objects in a circle around it) for predators and prey

    - (22 points/32 points) - Field of vision (cone and range) for predators, nothing for prey
    - (32 points/32 points) - Field of vision (cone and range) for predators, your choice of vision model for prey
- Report (**20 points**): Clearly state what you have implemented and detail the approaches you used in your solution. Use screen shots to illustrate these items.
    - State with whom you have had any collaboration with.
    - Write a detailed lab report as directed in the previous labs.
    - Provide a UML diagram (https://en.wikipedia.org/wiki/Class_diagram) of your code structure (class hierarchy). Also, implement using behavior trees, finite state machine, or GOAP?
    - Include any websites that you reference and anyone (including class mates) that you have had discussions regarding the lab.
    - Describe user controls, if any.
    - Document each completed extra credit (see below).
    - **Remember: Your solution must consist of source code originally authored by you alone. Do not copy or use altered source code from any other sources.**
- Extra credit (**Up to 10 points possible**):
    - Demo video (2 points) - Use Unity Record (or some other recording software) and create a demo video highlighting what you did, upload it to Youtube and include a link in your lab report. You must narrate in the video. Do not provide too many details that you will provide in your report. A good guideline is the video I showed in class: https://www.youtube.com/watch?v=QbUPfMXXQIY
    - Flocking (5 points) - Add flocking behavior to the prey (see boids)
    - Field of view (3 points) - Add a visualization that accurately shows the field-of-view of the agents. Note: it must be viewable in "Game" mode (e.g., Unity's scene display of a raycast doesn't count). To display the field-of-view, consider creating a triangle, cone, several line segments
    - Feel free to make multiple scenes with different scripts for the different additional features. Alternatively, create a larger scene with different rooms (possibly closed) that show off your features.

Submission

Submit your Unity package and lab report (PDF format only) in the Canvas box. Include your name in the filename for both files. Make sure the package includes all scripts and scenes (and images/textures and models if applicable) in it. As in previous labs, you can test to determine if your package can be graded by making a new Unity project, importing the package, opening the scene file, and running the solution. You must submit work that runs in order to receive credit.